Exemple d'utilisation du package levygof

Sommaire

Préparation **1**
Principe . . . 1

I - Une trajectoire **3**
Gamma . . . 3
Wiener . . . 5
Inverse gaussien . . . 5
Variance-gamma . . . 6
Meixner . . . 6
Un modèle faux . . . 6

II - Plusieurs trajectoires **8**

III - Cas particuliers **11**
Inspections régulières . . . 11
Rejet certain . . . 11
Un échantillon i.i.d. d'accroissements . . . 11
Messages . . . 12

IV - Références des tests **13**

Préparation

Installation des packages depuis GitHub : `levygof` pour les tests, `DegradSim` pour simuler des trajectoires. `levygof` utilise `levyfit` (estimation) et `levyBridge` (ponts), installés avec lui.

```
remotes::install_github("rytracetito/levy_packages", subdir = "levygof")
remotes::install_github("rytracetito/levy_packages", subdir = "DegradSim")
```

Principe

On veut savoir si des trajectoires de dégradation proviennent d'un processus de Lévy donné, dont les paramètres sont inconnus :

H_0 : les trajectoires sont des trajectoires indépendantes du processus `model`.

Les accroissements d'un processus de Lévy sur un même pas de temps sont i.i.d. La fonction `gof.levy()` procède ainsi :

- **inspections régulières** : les accroissements observés sont i.i.d. sous H_0, ils sont testés directement ;

- **inspections irrégulières** : les paramètres sont estimés par maximum de vraisemblance (`levyfit`), chaque trajectoire est rééchantillonnée sur une grille régulière par le pont du processus (`levyBridge`, méthode `"Bridge"`, `"ExtBridge"` ou `"MultiBridge"`), et les accroissements reconstruits, i.i.d. sous H_0, sont testés ;
- **gamma et inverse gaussien** : un accroissement observé négatif ou nul est impossible sous H_0, qui est alors rejetée sans calcul.

Les trois méthodes de rééchantillonnage :

- `"Bridge"` (Grall-Maës, 2012) : une grille par trajectoire, de pas t_N/N ; avec plusieurs trajectoires, les pas diffèrent et chaque trajectoire est testée seule ;
- `"ExtBridge"` : grille qui contient tous les instants d'inspection, de pas h commun à toutes les trajectoires ;
- `"MultiBridge"` : grille commune à toutes les trajectoires, de pas $h = \sum_i t_{i,max} / \sum_i m_i$ (durée totale divisée par le nombre total d'accroissements), avec l'option 1 (chaque trajectoire jusqu'à sa dernière inspection, recommandée) ou l'option 2 (toutes jusqu'au dernier instant global).

Avec `"ExtBridge"` et `"MultiBridge"`, les accroissements de toutes les trajectoires ont la même loi : ils sont aussi testés ensemble (groupe `all`).

Les modèles : `"HGP"` (gamma), `"HWP"` (Wiener), `"InvGaus"` (inverse gaussien), `"VG"` (variance-gamma), `"Meixner"`. Les tests de chaque modèle :

```
str(tests_disponibles())
```

```
## List of 5
##  $ HGP    : chr [1:7] "HME1" "BE" "VGE" "KS" ...
##  $ HWP    : chr [1:6] "AD" "CvM" "SW" "Lilliefors" ...
##  $ InvGaus: chr [1:9] "ABEV1" "ABEV2" "HK1" "HK2" ...
##  $ VG     : chr [1:4] "KS" "CvM" "AD" "Watson"
##  $ Meixner: chr [1:4] "KS" "CvM" "AD" "Watson"
```

Le pont et le bootstrap sont aléatoires : `set.seed()` rend les résultats reproductibles.

I - Une trajectoire

Une trajectoire observée à 31 instants irréguliers, donnés avec une décimale.

```
set.seed(2026)
t <- c(0, cumsum(round(runif(30, 0.5, 3.5), 1)))
```

Gamma

```
set.seed(1)
x <- sim_gamma(t, a = 2, b = 3)
res <- gof.levy(x, "HGP", B = 100)
res
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 1 | Increments tested: 30 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "Bridge")
## Time step: 1.537
## Estimated parameters of the process: a = 2.281, b = 3.588
##
##    test statistic p.value critical.value         decision
##    HME1  0.006779  0.4554         0.0350 do not reject H0
##      BE  0.035250  0.5050         0.1461 do not reject H0
##     VGE  1.205000  0.1485         1.7640 do not reject H0
##      KS  0.094790  0.7624         0.1574 do not reject H0
##     CvM  0.048070  0.5941         0.1317 do not reject H0
##      AD  0.325300  0.4752         0.6376 do not reject H0
##  Watson  0.044500  0.5446         0.1183 do not reject H0
```

Le résultat contient aussi les paramètres estimés, les pseudo-accroissements et la trajectoire rééchantillonnée :

```
res$parameters
```

```
##        a        b
## 2.281135 3.588177
```

```
head(res$increments)
```

```
##   id     step        dx
## 1  1 1.536667 0.7363521
## 2  1 1.536667 1.6595409
## 3  1 1.536667 0.9125281
## 4  1 1.536667 1.7290214
## 5  1 1.536667 0.8494291
## 6  1 1.536667 1.8601805
```

```
res$results[, c("test", "statistic", "p.value", "critical.value", "decision")]
```

```
##     test   statistic   p.value critical.value         decision
## 1   HME1 0.006778983 0.4554455     0.03500468 do not reject H0
## 2     BE 0.035245488 0.5049505     0.14607218 do not reject H0
## 3    VGE 1.205404244 0.1485149     1.76353434 do not reject H0
## 4     KS 0.094787886 0.7623762     0.15737687 do not reject H0
## 5    CvM 0.048070047 0.5940594     0.13173973 do not reject H0
## 6     AD 0.325303644 0.4752475     0.63758720 do not reject H0
```

```
## 7 Watson 0.044495416 0.5445545     0.11828074 do not reject H0
```

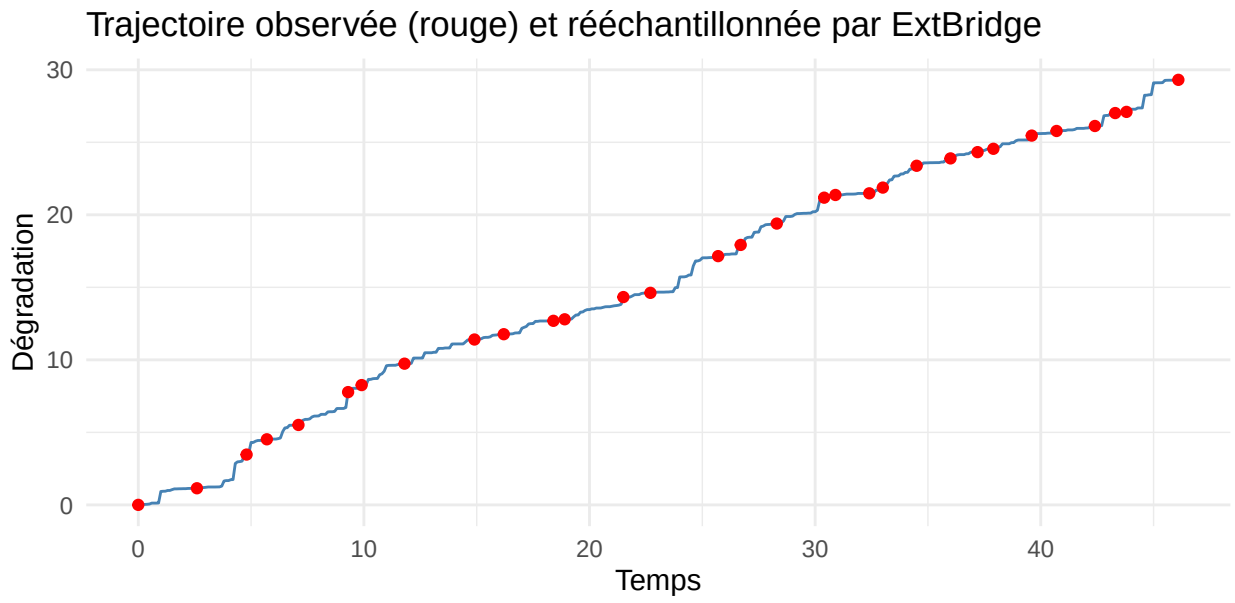
Pour les tests calculés par bootstrap paramétrique, le tableau donne la p-valeur et la valeur critique au niveau α (quantile d'ordre $1-\alpha$ des statistiques bootstrap) : H_0 est rejetée quand la p-valeur est inférieure à α, c'est-à-dire quand la statistique dépasse la valeur critique. Quand elle n'est pas disponible (tests à p-valeur exacte ou asymptotique comme ceux de normalité, tests de `gofIG` qui ne donnent que la p-valeur), la valeur critique vaut `NA`.

La même trajectoire avec la méthode `"ExtBridge"` : la grille, de pas 0.1, contient toutes les inspections.

```r
set.seed(1)
gof.levy(x, "HGP", test = c("VGE", "KS", "AD"), method = "ExtBridge", B = 100)
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 1 | Increments tested: 461 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "ExtBridge")
## Time step: 0.1
## Estimated parameters of the process: a = 2.281, b = 3.588
##
##  test statistic p.value critical.value         decision
##   VGE   0.21250 0.88120        2.15000 do not reject H0
##    KS   0.04265 0.09901        0.04596 do not reject H0
##    AD   0.36110 0.57430        1.04700 do not reject H0
```

```r
r_ext <- gof.levy(x, "HGP", test = "AD", method = "ExtBridge", B = 100)
ggplot() +
  geom_line(data = r_ext$bridge[[1]], aes(t, X), colour = "steelblue") +
  geom_point(data = data.frame(t = x[, 1], X = x[, 2]), aes(t, X), colour = "red") +
  labs(title = "Trajectoire observée (rouge) et rééchantillonnée par ExtBridge",
       x = "Temps", y = "Dégradation") +
  theme_minimal()
```



Wiener

```
set.seed(2)
xw <- sim_wiener(t, mu = 0.5, sigma = 1.5)
gof.levy(xw, "HWP")
```

```
##
## Goodness-of-fit test of the model HWP
## Trajectories: 1 | Increments tested: 30 | Level: 0.05 | B: 200
## Inspections: irregular (bridge, method "Bridge")
## Time step: 1.537
## Estimated parameters of the process: mu = 0.6929, sigma = 1.748
##
##              test statistic p.value         decision
##                AD   0.28450  0.6054 do not reject H0
##               CvM   0.04173  0.6369 do not reject H0
##                SW   0.97070  0.5580 do not reject H0
##        Lilliefors   0.08728  0.8102 do not reject H0
##                JB   1.79000  0.4087 do not reject H0
##  DAgostinoPearson   3.16000  0.2059 do not reject H0
```

```
gof.levy(xw, "HWP", method = "ExtBridge")
```

```
##
## Goodness-of-fit test of the model HWP
## Trajectories: 1 | Increments tested: 461 | Level: 0.05 | B: 200
## Inspections: irregular (bridge, method "ExtBridge")
## Time step: 0.1
## Estimated parameters of the process: mu = 0.6929, sigma = 1.748
##
##              test statistic p.value         decision
##                AD   0.24870  0.7476 do not reject H0
##               CvM   0.04475  0.5937 do not reject H0
##                SW   0.99830  0.9433 do not reject H0
##        Lilliefors   0.02714  0.5623 do not reject H0
##                JB   0.38920  0.8232 do not reject H0
##  DAgostinoPearson   0.43330  0.8052 do not reject H0
```

Inverse gaussien

```
set.seed(3)
xi <- sim_ig(t, mu = 1, eta = 3)
gof.levy(xi, "InvGaus", test = c("HK2", "KS", "AD", "VGE_ratio"), method = "ExtBridge", B = 100)
```

```
##
## Goodness-of-fit test of the model InvGaus
## Trajectories: 1 | Increments tested: 461 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "ExtBridge")
## Time step: 0.1
## Estimated parameters of the process: mu = 1.068, eta = 4.48
##
##       test statistic p.value         decision
##        HK2   0.16830  0.1200 do not reject H0
```

```
##         KS   0.03665  0.3200 do not reject H0
##         AD   0.59540  0.2600 do not reject H0
##  VGE_ratio  -1.02400  0.3058 do not reject H0
```

Variance-gamma

Les tests KS, CvM, AD et Watson du modèle VG ajustent la loi par maximum de vraisemblance (`levyfit`) à chaque réplication bootstrap : ils sont plus lents.

```
set.seed(4)
xv <- sim_vg(t, theta = 0.5, sigma = 1.5, nu = 0.5)
gof.levy(xv, "VG", test = c("KS", "AD"), method = "ExtBridge", B = 50)
```

```
##
## Goodness-of-fit test of the model VG
## Trajectories: 1 | Increments tested: 461 | Level: 0.05 | B: 50
## Inspections: irregular (bridge, method "ExtBridge")
## Time step: 0.1
## Estimated parameters of the process: theta = 0.471, sigma = 1.566, nu = 1.391
##
##  test statistic p.value critical.value         decision
##    KS   0.03224   0.451        0.05197 do not reject H0
##    AD   0.45380   0.451        1.36100 do not reject H0
```

Meixner

Avec peu d'accroissements, l'estimateur du maximum de vraisemblance du processus de Meixner peut ne pas exister (maximum sur le bord, vers une loi gamma décalée ou gaussienne). Sur les données d'origine, `gof.levy()` s'arrête alors avec un message ; sur les pseudo-accroissements, le test est marqué `not computed`. Avec 60 inspections, cela arrive encore souvent : le modèle de Meixner demande beaucoup de données.

```
set.seed(1)
t60 <- c(0, cumsum(round(runif(60, 0.5, 3.5), 1)))
set.seed(7)
xm <- sim_meixner(t60, a = 1, b = 1, d = 0.5, m = 0.2)
gof.levy(xm, "Meixner", test = "AD", B = 30)
```

```
##
## Goodness-of-fit test of the model Meixner
## Trajectories: 1 | Increments tested: 60 | Level: 0.05 | B: 30
## Inspections: irregular (bridge, method "Bridge")
## Time step: 2.035
## Estimated parameters of the process: a = 0.8886, b = 1.994, d = 0.3592, m = 0.04248
##
##  test statistic p.value critical.value         decision
##    AD    0.3293  0.3333         0.3748 do not reject H0
```

Un modèle faux

Une trajectoire gamma testée comme un processus inverse gaussien, puis comme un processus de Wiener :

```r
set.seed(6)
xg <- sim_gamma(t, a = 0.5, b = 0.25)
gof.levy(xg, "InvGaus", test = c("KS", "AD"), B = 100)
```

```
##
## Goodness-of-fit test of the model InvGaus
## Trajectories: 1 | Increments tested: 30 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "Bridge")
## Time step: 1.537
## Estimated parameters of the process: mu = 2.21, eta = 0.06989
##
##  test statistic p.value  decision
##    KS     0.262       0 reject H0
##    AD     3.333       0 reject H0
```

```r
gof.levy(xg, "HWP", test = c("SW", "AD"))
```

```
##
## Goodness-of-fit test of the model HWP
## Trajectories: 1 | Increments tested: 30 | Level: 0.05 | B: 200
## Inspections: irregular (bridge, method "Bridge")
## Time step: 1.537
## Estimated parameters of the process: mu = 2.21, sigma = 3.186
##
##  test statistic   p.value  decision
##    SW    0.8665 0.0014020 reject H0
##    AD    1.4370 0.0008314 reject H0
```

II - Plusieurs trajectoires

Cinq trajectoires gamma, chacune avec ses propres instants d'inspection (liste de data frames).

```
set.seed(7)
plan <- lapply(1:5, function(k) c(0, cumsum(round(runif(20, 0.5, 3.5), 1))))
xs <- lapply(plan, function(tk) {
  data.frame(t = tk, X = c(0, cumsum(rgamma(20, 2 * diff(tk), 3))))
})
```

Avec `"Bridge"`, chaque trajectoire a son pas t_N/N : les pseudo-accroissements de deux trajectoires n'ont pas la même loi, et chaque trajectoire est testée seule. Avec plusieurs tests au niveau 5 %, un rejet isolé peut arriver par hasard même sous H_0.

```
set.seed(8)
r_b <- gof.levy(xs, "HGP", test = c("VGE", "AD"), B = 100)
r_b
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 5 | Increments tested: 100 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "Bridge")
## Time step: from 1.735 to 2.225
## Estimated parameters of the process: a = 3.27, b = 4.746
##
##  group test statistic p.value critical.value         decision
##      1  VGE    0.5127 0.57430         1.8330 do not reject H0
##      1   AD    0.2390 0.88120         0.8564 do not reject H0
##      2  VGE    0.5202 0.57430         2.2430 do not reject H0
##      2   AD    0.4074 0.43560         0.8679 do not reject H0
##      3  VGE    1.7480 0.06931         1.8360 do not reject H0
##      3   AD    0.8147 0.02970         0.6882        reject H0
##      4  VGE    0.3233 0.75250         2.3160 do not reject H0
##      4   AD    0.2847 0.75250         0.7898 do not reject H0
##      5  VGE    0.4631 0.65350         2.0690 do not reject H0
##      5   AD    0.5336 0.19800         0.7278 do not reject H0
```

Avec `"ExtBridge"`, toutes les grilles ont le même pas : les pseudo-accroissements des cinq trajectoires sont aussi testés ensemble (groupe `all`).

```
set.seed(8)
r_e <- gof.levy(xs, "HGP", test = c("VGE", "AD"), method = "ExtBridge", B = 100)
r_e
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 5 | Increments tested: 2029 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "ExtBridge")
## Time step: 0.1
## Estimated parameters of the process: a = 3.27, b = 4.746
##
##  group test statistic p.value critical.value         decision
##    all  VGE    0.9114 0.53470         2.6090 do not reject H0
##    all   AD    0.7084 0.10890         0.8011 do not reject H0
##      1  VGE    1.4190 0.19800         2.1960 do not reject H0
##      1   AD    0.2616 0.80200         0.9489 do not reject H0
```

```
##      2  VGE    0.8826 0.54460         2.3060 do not reject H0
##      2   AD    0.4466 0.31680         0.8928 do not reject H0
##      3  VGE    0.2489 0.80200         2.3630 do not reject H0
##      3   AD    0.1265 0.99010         0.9567 do not reject H0
##      4  VGE    1.1620 0.41580         2.4750 do not reject H0
##      4   AD    0.7687 0.09901         0.8676 do not reject H0
##      5  VGE    1.0160 0.39600         2.5340 do not reject H0
##      5   AD    0.5394 0.21780         1.0220 do not reject H0
```

Avec `"MultiBridge"`, toutes les trajectoires sont placées sur une grille commune $t'_k = kh$, de pas

$$h = \frac{t_{1,max} + \cdots + t_{n,max}}{m_1 + \cdots + m_n},$$

la durée totale observée divisée par le nombre total d'accroissements (pour une seule trajectoire, $h = t_N/N$: c'est la méthode `"Bridge"`). La grille ne contient pas forcément les instants d'inspection. Elle est parcourue de gauche à droite : un point de grille égal à une inspection garde la valeur observée, les autres sont tirés dans le pont du processus entre la dernière valeur connue à gauche et l'inspection suivante, avec les paramètres estimés par `levyfit`.

- **Option 1** (par défaut, recommandée) : chaque trajectoire i est rééchantillonnée sur les points $kh \leq t_{i,max}$; le petit bout entre le dernier point de grille et $t_{i,max}$ n'est pas utilisé.
- **Option 2** : toutes les trajectoires vont jusqu'au dernier instant global $\max_i t_{i,max}$. Jusqu'à $t_{i,max}$, comme l'option 1 ; au-delà, il n'y a plus d'observation ni de pont : chaque valeur est la précédente plus un accroissement du processus tiré avec **tous** les paramètres estimés (ici a et b), en partant de la dernière observation. Ces valeurs ne viennent pas des données et font baisser la puissance.

Tous les pseudo-accroissements ont la loi de X_h sous H_0 : tests par trajectoire et test sur l'ensemble (groupe `all`).

```
set.seed(8)
r_m <- gof.levy(xs, "HGP", test = c("VGE", "AD"), method = "MultiBridge", B = 100)
r_m
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 5 | Increments tested: 97 | Level: 0.05 | B: 100
## Inspections: irregular (bridge, method "MultiBridge, option 1")
## Time step: 2.029
## Estimated parameters of the process: a = 3.27, b = 4.746
##
##  group test statistic p.value critical.value         decision
##    all  VGE    0.4381  0.7129         2.1510 do not reject H0
##    all   AD    0.3867  0.3762         0.7854 do not reject H0
##      1  VGE    0.9414  0.3069         2.0690 do not reject H0
##      1   AD    0.2916  0.6733         0.8127 do not reject H0
##      2  VGE    0.5869  0.4950         2.0440 do not reject H0
##      2   AD    0.9118  0.0297         0.7243        reject H0
##      3  VGE    1.2690  0.2673         1.9140 do not reject H0
##      3   AD    0.3144  0.6436         0.6903 do not reject H0
##      4  VGE    1.0210  0.2970         2.0710 do not reject H0
##      4   AD    0.6272  0.1287         0.7834 do not reject H0
##      5  VGE    0.6769  0.4554         1.9270 do not reject H0
##      5   AD    0.3112  0.6139         0.7160 do not reject H0
```

```
table(r_m$increments$id)
```

```
##
##  1  2  3  4  5
## 17 21 21 20 18
```

L'option 2 prolonge les trajectoires les plus courtes jusqu'au dernier instant global, avec les paramètres estimés : toutes ont alors le même nombre d'accroissements, mais les valeurs prolongées ne viennent pas des données.

```
set.seed(8)
r_m2 <- gof.levy(xs, "HGP", test = c("VGE", "AD"), method = "MultiBridge",
                 option = 2, B = 100)
table(r_m2$increments$id)
```

```
##
##  1  2  3  4  5
## 21 21 21 21 21
```

Les trajectoires simulées par `DegradSim` sur un même plan (matrice) ou sur des plans différents (liste) s'utilisent directement :

```
set.seed(9)
m <- sim_wiener(t, mu = 0.5, sigma = 1.5, n_traj = 4)
gof.levy(m, "HWP", test = "SW")
```

```
##
## Goodness-of-fit test of the model HWP
## Trajectories: 4 | Increments tested: 120 | Level: 0.05 | B: 200
## Inspections: irregular (bridge, method "Bridge")
## Time step: 1.537
## Estimated parameters of the process: mu = 0.4069, sigma = 1.402
##
##  group test statistic p.value         decision
##    all   SW    0.9914  0.6671 do not reject H0
##      1   SW    0.9545  0.2231 do not reject H0
##      2   SW    0.9655  0.4244 do not reject H0
##      3   SW    0.9754  0.6931 do not reject H0
##      4   SW    0.9704  0.5490 do not reject H0
```

III - Cas particuliers

Inspections régulières

Les accroissements observés sont testés directement, sans estimation ni pont (l'origine des temps est libre).

```
set.seed(10)
t_reg <- seq(10, 70, by = 2)
x_reg <- data.frame(t = t_reg, X = cumsum(c(5, rgamma(30, 2 * 2, 3))))
gof.levy(x_reg, "HGP", test = c("VGE", "AD"), B = 100)
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 1 | Increments tested: 30 | Level: 0.05 | B: 100
## Inspections: regular (observed increments)
## Time step: 2
##
##  test statistic p.value critical.value         decision
##   VGE    0.9035  0.4257          1.983 do not reject H0
##    AD    0.4514  0.2277          0.820 do not reject H0
```

Rejet certain

Une observation en baisse est impossible pour un processus gamma :

```
x_bas <- data.frame(t = t, X = x[, 2])
x_bas$X[12] <- x_bas$X[11] - 0.1
gof.levy(x_bas, "HGP", test = c("KS", "AD"))
```

```
##
## Goodness-of-fit test of the model HGP
## Trajectories: 1 | Increments tested: 30 | Level: 0.05 | B: 200
## Inspections: observed increment <= 0 under an increasing model, H0 rejected for sure
##
##  test statistic p.value  decision
##    KS        NA       0 reject H0
##    AD        NA       0 reject H0
##                                                                    message
##  1 negative or zero increment(s): impossible under H0 (increasing process)
##  1 negative or zero increment(s): impossible under H0 (increasing process)
```

Un échantillon i.i.d. d'accroissements

`test_adequation()` applique les tests à des accroissements déjà disponibles :

```
set.seed(11)
test_adequation(rnorm(50, 1, 2), "HWP")
```

```
##              test  statistic   p.value critical.value         decision
## 1              AD 0.25413580 0.7175494             NA do not reject H0
## 2             CvM 0.03822047 0.7077303             NA do not reject H0
## 3              SW 0.98625005 0.8237406             NA do not reject H0
## 4      Lilliefors 0.07611195 0.6654508             NA do not reject H0
## 5              JB 0.59377008 0.7431294             NA do not reject H0
```

```
## 6 DAgostinoPearson 0.43612149 0.8040766             NA do not reject H0
```

```
test_adequation(rgamma(40, 4, 3), "HGP", test = c("VGE", "AD"), alpha = 0.1, B = 100)
```

```
##   test statistic   p.value critical.value         decision
## 1  VGE  1.091795 0.3267327       2.127591 do not reject H0
## 2   AD  0.368970 0.3960396       0.722369 do not reject H0
```

Messages

Un test qui échoue n'arrête pas les autres ; les données incorrectes sont refusées avec un message clair.

```
test_adequation(c(1, 1, 1, 1, 1), "HWP", test = c("SW", "JB"))
```

```
##   test statistic p.value critical.value     decision
## 1   SW        NA      NA             NA not computed
## 2   JB       NaN     NaN             NA not computed
##                                   message
## 1 toutes les valeurs de 'x' sont identiques
## 2
```

```
try(gof.levy(data.frame(t = c(0, 2, 1, 3, 4), X = 1:5), "HGP"))
```

```
## Error : trajectory 1: the inspection times must be strictly increasing
```

```
try(gof.levy(x, "Poisson"))
```

```
## Error : unknown model 'Poisson'; available models: HGP, HWP, InvGaus, VG, Meixner
```

IV - Références des tests

Les références de chaque test sont données dans l'aide :

```
?tests_disponibles
```

- gamma : Henze, Meintanis et Ebner (2012) ; Betsch et Ebner (2019) ; Villaseñor et González-Estrada (2015) ;
- normale (Wiener) : Shapiro et Wilk (1965) ; Lilliefors (1967) ; Stephens (1986) ; Jarque et Bera (1987) ; D'Agostino et Pearson (1973) ;
- inverse gaussienne : Allison, Betsch, Ebner et Visagie (2022) ; Henze et Klar (2002) ; Villaseñor et González-Estrada (2015) ;
- variance-gamma et Meixner : statistiques EDF (Stephens, 1986) et bootstrap paramétrique (Stute, González Manteiga et Presedo Quindimil, 1993) ;
- méthode du pont : Grall-Maës (2012).